

Building LLM Apps with the OpenAI Responses API & LangChain v1

Lucas Soares

06-10-2026

Quick 'Interactivity Notes'

- The presentation will be organized into the following structure:
- **Presentation Block**
- **Notebook Demo**
- **Quick Q&A + Summary**
- **Optional Exercise** During Q&A (for those that don't have questions and want to try something out)
- Repeat

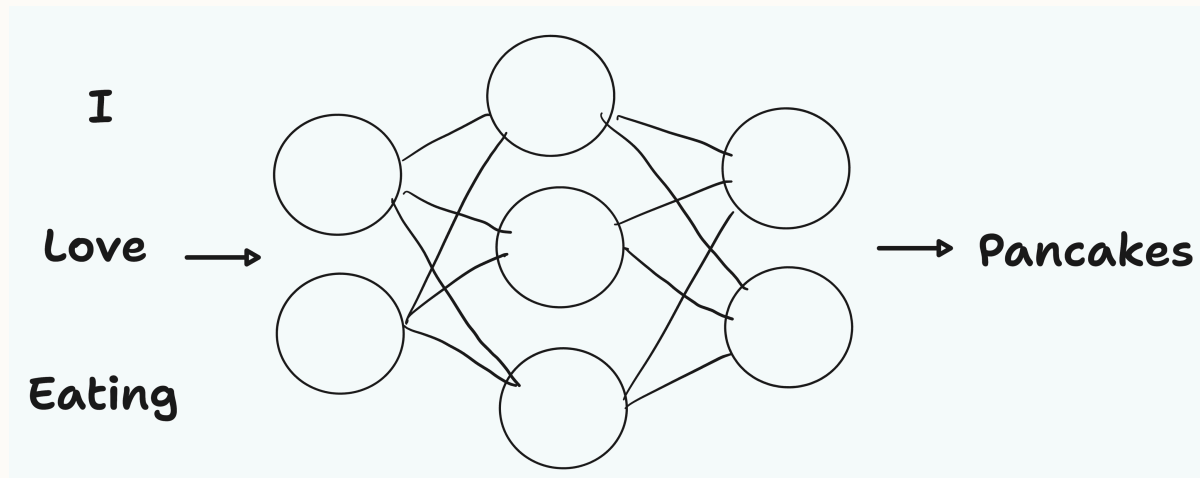
Today's Roadmap

1. **LLM Foundations** — what they are, how transformers work
2. **OpenAI Responses API** — the new core API, model lineup, reasoning effort, hosted tools
3. **Prompt Basics** — anatomy of a great prompt
4. **Fine-Tuning** — when and how to specialize a model
5. **LangChain v1 + LangGraph** — models, structured output, agents with `create_agent`
6. **RAG** — chat over your documents
7. **What's deprecated in 2026** — so your code doesn't rot

Large Language Models

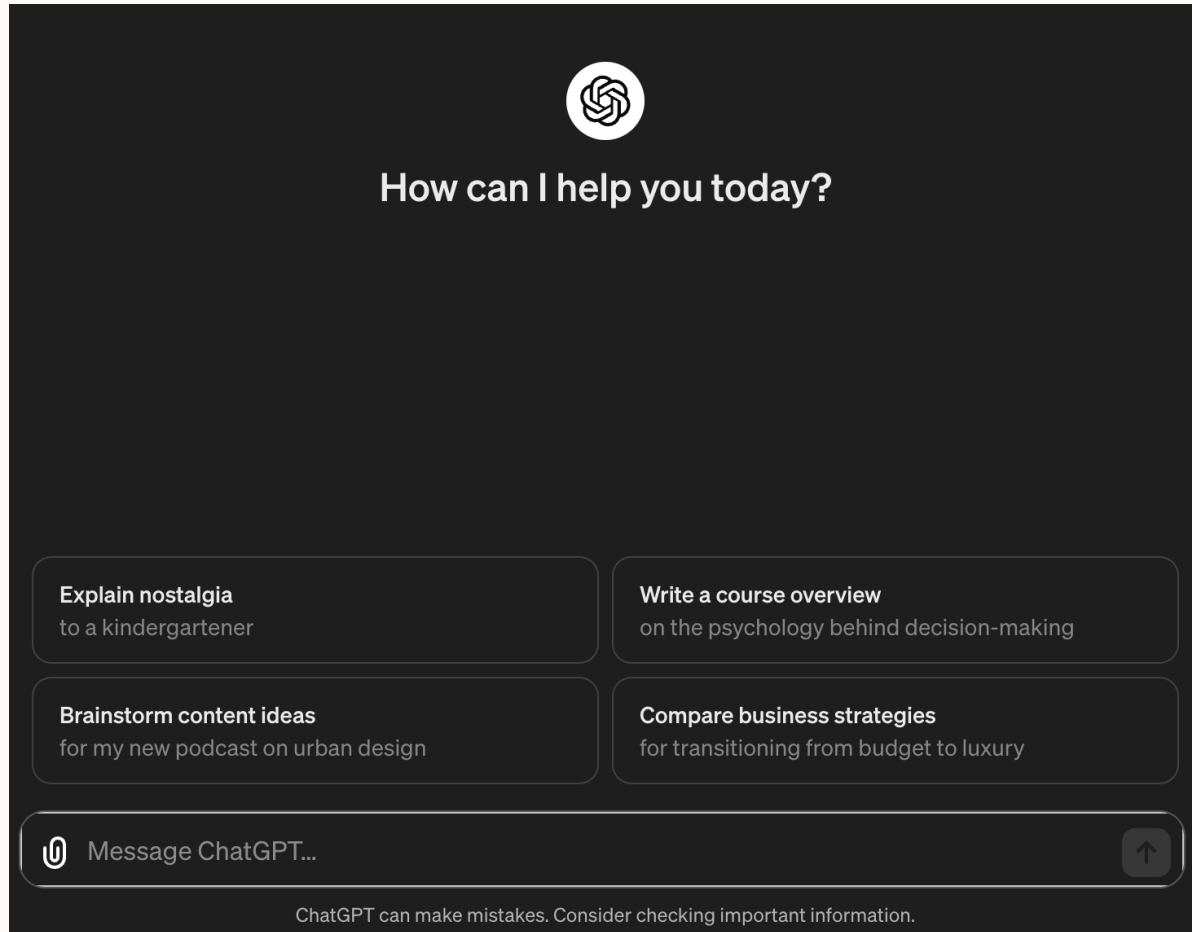
Large Language Models

Large Language Models Predict the Next Word



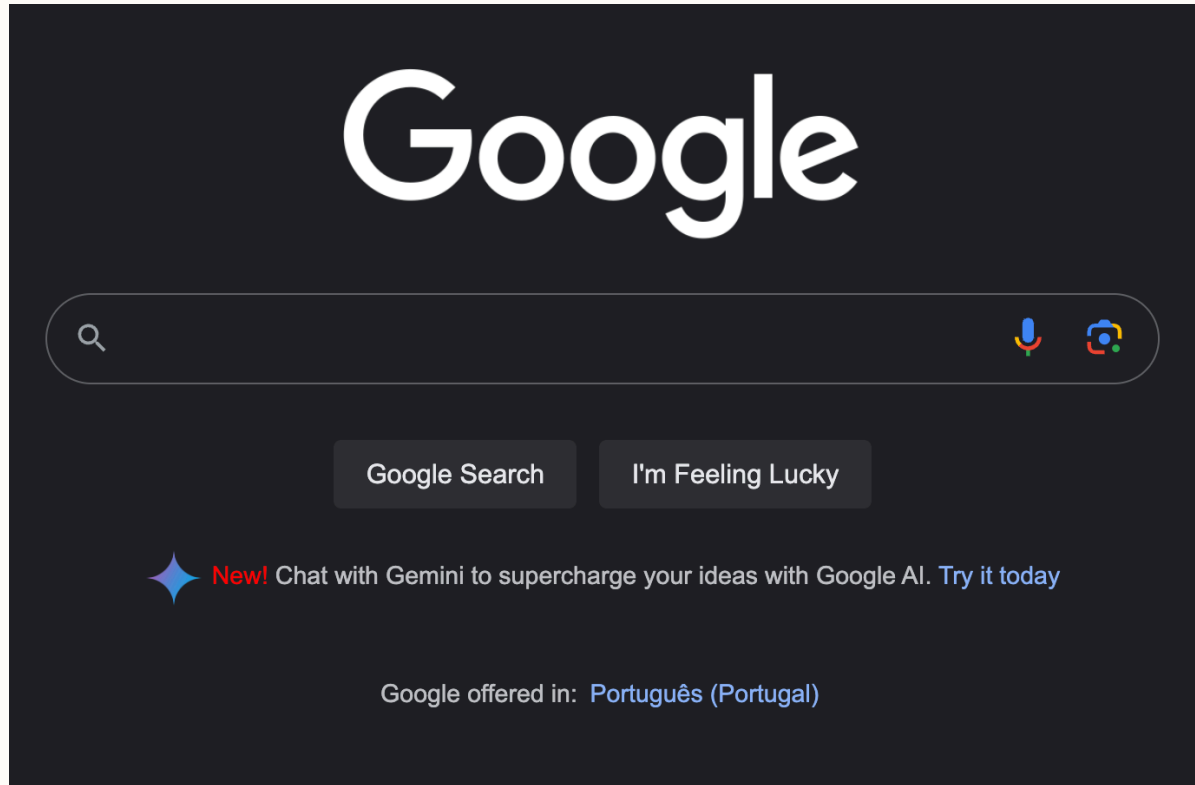
Applications of Large Language Models

- Conversational ChatBots



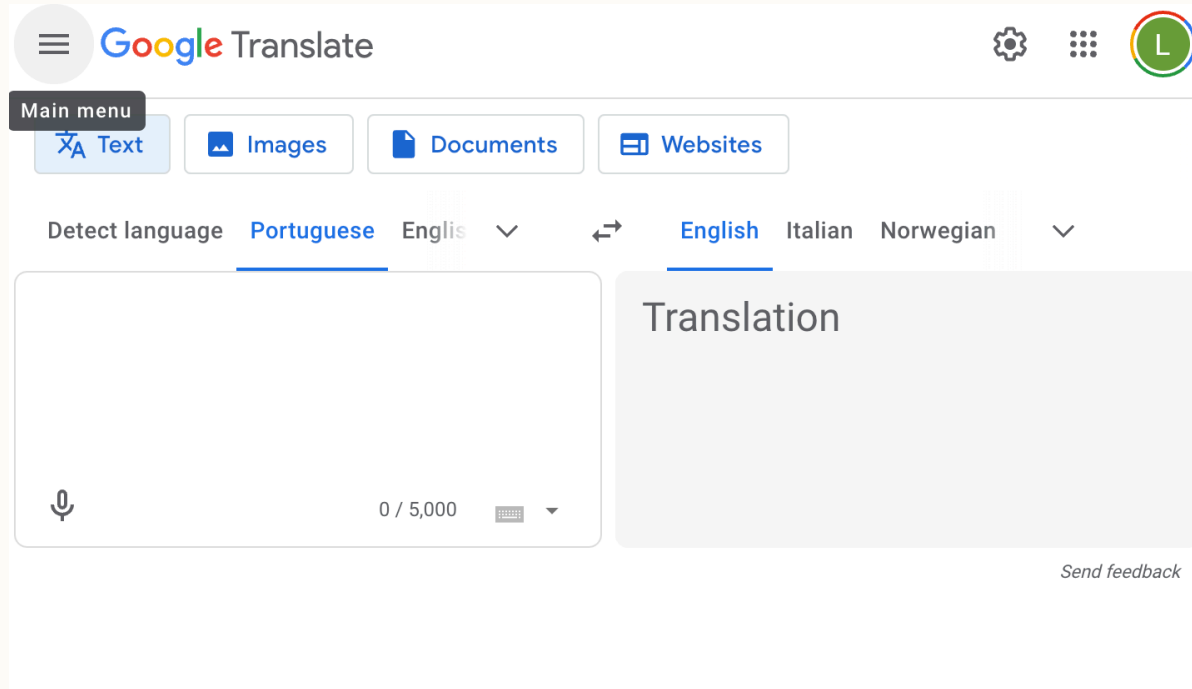
Applications of Large Language Models

- Search Engines



Applications of Large Language Models

- Translation



- And so much more from Q&A over PDFs to personalized tutoring.

Behind all of these applications is a powerful architecture:
Transformers

Transformer Process

Input: "I love eating _____"

Tokenize: I love eating

Embedding: [0.78, 1.45, 0.35]

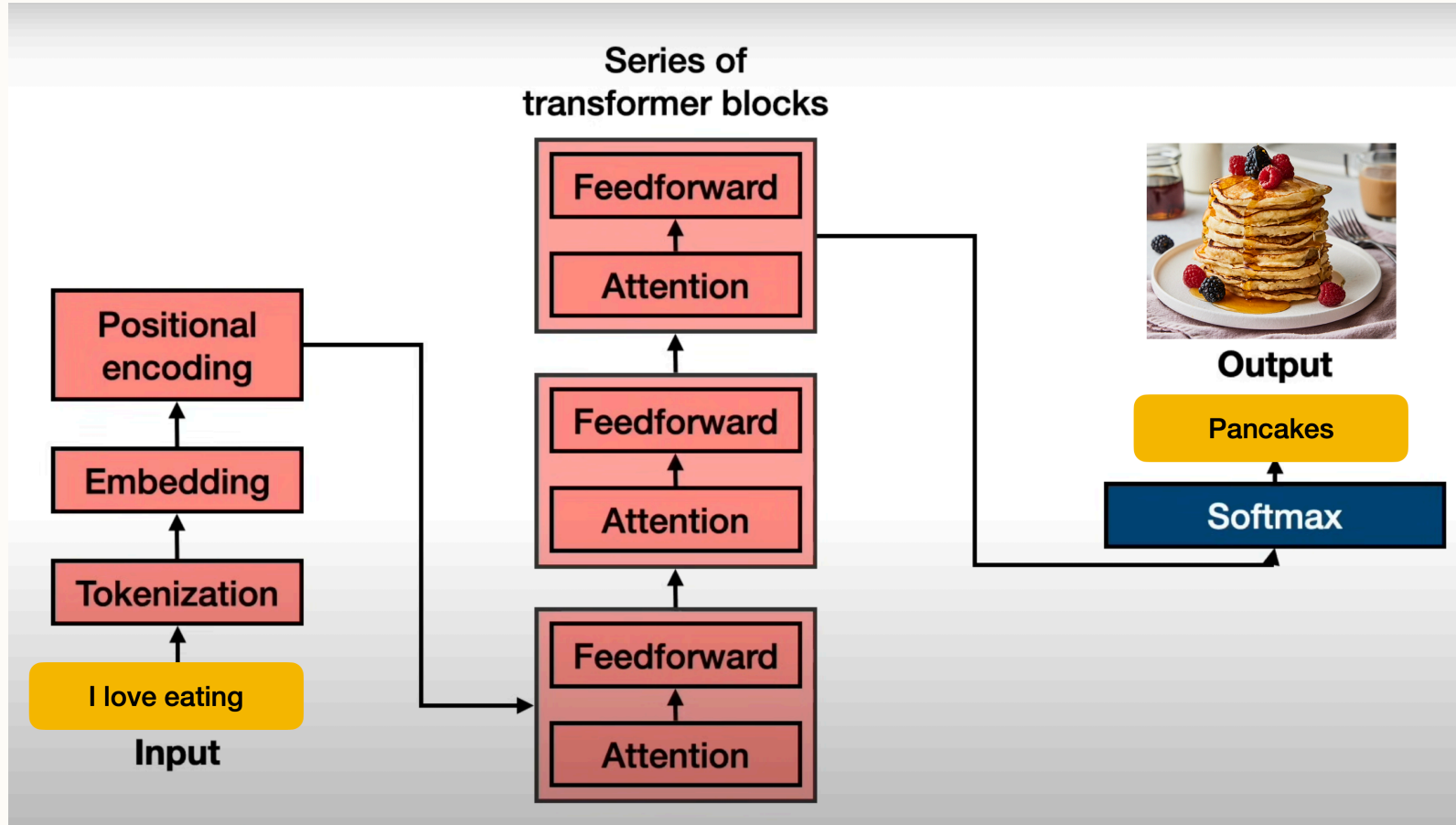
Positional Encoding: 1. I 2. love 3. eating

Attention: Understands context

"I" (Lucas) and "love": things that Lucas loves, cross-referenced with what Lucas eats



Transformers



[1][What are Transformers and How do They Work? By Serrano Academy](#)

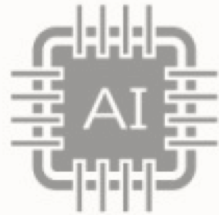
Benefits of Large Language Models

- Multi-task Capability
 - LLMs find applications in content generation, question answering, translation, tutoring, and personal assistants.
- Fine-tuning
 - LLMs can usually be fine tuned with a relatively small amount of data, making them adaptable to a wide range of tasks.
- Scalability
 - LLMs demonstrate excellent scalability to very large capacity networks and huge datasets.

Limitations and Ethical Considerations

LLMs are far from perfect

- **Knowledge Limit:** LLMs have a cutoff point for their knowledge.
- **Understanding Limit:** LLMs do not understand text in the same way humans do. They don't have beliefs or desires; they simply predict what comes next based on their training.
- **Misuse:** LLMs can hallucinate and produce false or harmful content
- **Reproducibility:** Unpredictability of LLM behavior. Watkins 2023
- **Data Privacy and Bias:** Ethical considerations should extend to the acquisition of data for training additional models. Models may have biases; their use should be transparent and biases mitigated. Watkins 2023



Applications of LLMs

Content generation Example - DEMO

Demo App - Panda Letters

Q&A & Summary

- **LLMs predict the next word** and generate human-like text.
- **LLMs power diverse applications** including conversational chatbots, search engines, and translation services, demonstrating their versatility in handling various language tasks.
- **Transformer architecture** processes text in parallel, utilizing mechanisms like tokenization, embedding, positional encoding, and attention to understand context and generate relevant outputs.
- **Advantages of LLMs:** LLMs are multi-taskers capable of content generation, question answering, and more, with the ability to be fine-tuned for specific tasks and scale effectively to handle large datasets and complex models.
- **Benefits and Limitations:** Despite their benefits (multi-tasking, fine-tuning etc.), LLMs have limitations, including a fixed knowledge base, lack of true understanding, potential for misuse, and ethical concerns around data privacy and inherent biases.
- Optional Exercise
 - Set up your OpenAI API key and prepare for the practical demos

Break

OpenAI Responses API

The single API for models + tools + state

Introduction to the OpenAI Responses API

- The **Responses API** is OpenAI's core API: one endpoint for text generation, reasoning, hosted tools, and multi-turn state.
- Basic structure of calling the API:

```
from openai import OpenAI
client = OpenAI()

def get_response(prompt_question):
    response = client.responses.create(
        model="gpt-5.4-mini",
        instructions="You are a helpful research and programming assistant",
        input=prompt_question,
    )
    return response.output_text
```

- Required parameters: `model`, `input`
- `instructions` replaces the old system message; `response.output_text` gives you the answer directly.

[2][OpenAI Docs - Responses API](#)

Responses vs. Chat Completions

Chat Completions (legacy)

```
completion = client.chat.completions.create(
    model="gpt-5.4-mini",
    messages=[
        {"role": "system",
         "content": "You are helpful."},
        {"role": "user",
         "content": "Hi!"},
    ],
)

print(completion.choices[0]
      .message.content)
```

Responses (current)

```
response = client.responses.create(
    model="gpt-5.4-mini",
    instructions="You are helpful.",
    input="Hi!",
)

print(response.output_text)

# multi-turn without resending
# the whole transcript:
client.responses.create(
    model="gpt-5.4-mini",
    previous_response_id=response.id,
    input="Tell me more!")
```

- OpenAI reports a **40–80% improvement in cache utilization** vs Chat Completions in internal tests → lower latency and cost.

[3] [Migrate to the Responses API](#)

The 2026 Model Lineup

Model	One-line use case
<code>gpt-5.5</code>	Flagship: hardest coding, agentic, and professional work
<code>gpt-5.5-pro</code>	Maximum reasoning depth for the toughest problems (defaults to high effort)
<code>gpt-5.4</code>	Affordable previous-gen workhorse for coding & professional work
<code>gpt-5.4-mini</code>	Fast + cheap: coding, computer use, and subagents
<code>gpt-5.4-nano</code>	Ultra-low latency/cost: classification, autocomplete, simple extraction

In this course we'll use `gpt-5.4-mini` for most examples — best price/performance for learning and prototyping.

[4][OpenAI Docs - Models](#)

Controlling Thinking: reasoning_effort

```
response = client.responses.create(  
    model="gpt-5.5",  
    reasoning={"effort": "high"},  
    input="Find the bug in this concurrency code...",  
)
```

- **low** — efficient reasoning when tool-use/planning still matters; latency-sensitive apps.
- **medium** — the default: balanced quality, latency, and cost for most workflows.
- **high** — complex agentic tasks, deep planning and debugging, when latency is less critical.
- **xhigh** — hardest async tasks: deep research, frontier problem-solving — let it cook.

 *Higher is not automatically better: with vague instructions, more effort can mean overthinking. Increase only when evals show gains.*

[5] [OpenAI Docs - Using GPT-5.5](#)

Hosted Tools: Batteries Included

One parameter turns the model into an agent — OpenAI runs the tools for you:

```
response = client.responses.create(  
    model="gpt-5.4-mini",  
    tools=[{"type": "web_search"}],  
    input="What changed in LangChain v1?",  
)
```

Tool	What it does
web_search	Up-to-date answers from the web, with citations
file_search	Semantic search over your uploaded files (hosted RAG)
code_interpreter	Writes & runs Python in a sandbox (data analysis, math)
image_generation	Generates / edits images inside the same conversation
computer_use	Agentic control of a screen: clicks, typing, navigation
mcp	Connect any remote MCP server as a tool source

[6] [OpenAI Docs - Using tools](#)

Notebook Demo 1.0 - Intro to the Responses API & Prompt Basics

Prompt Basics



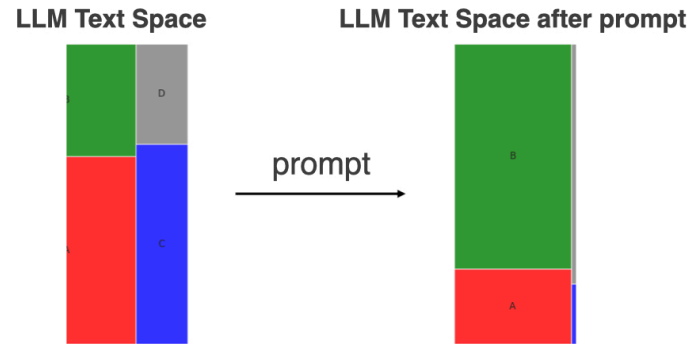
A prompt is a piece of text that conveys to the LLM the user's intention.

Prompt Basics

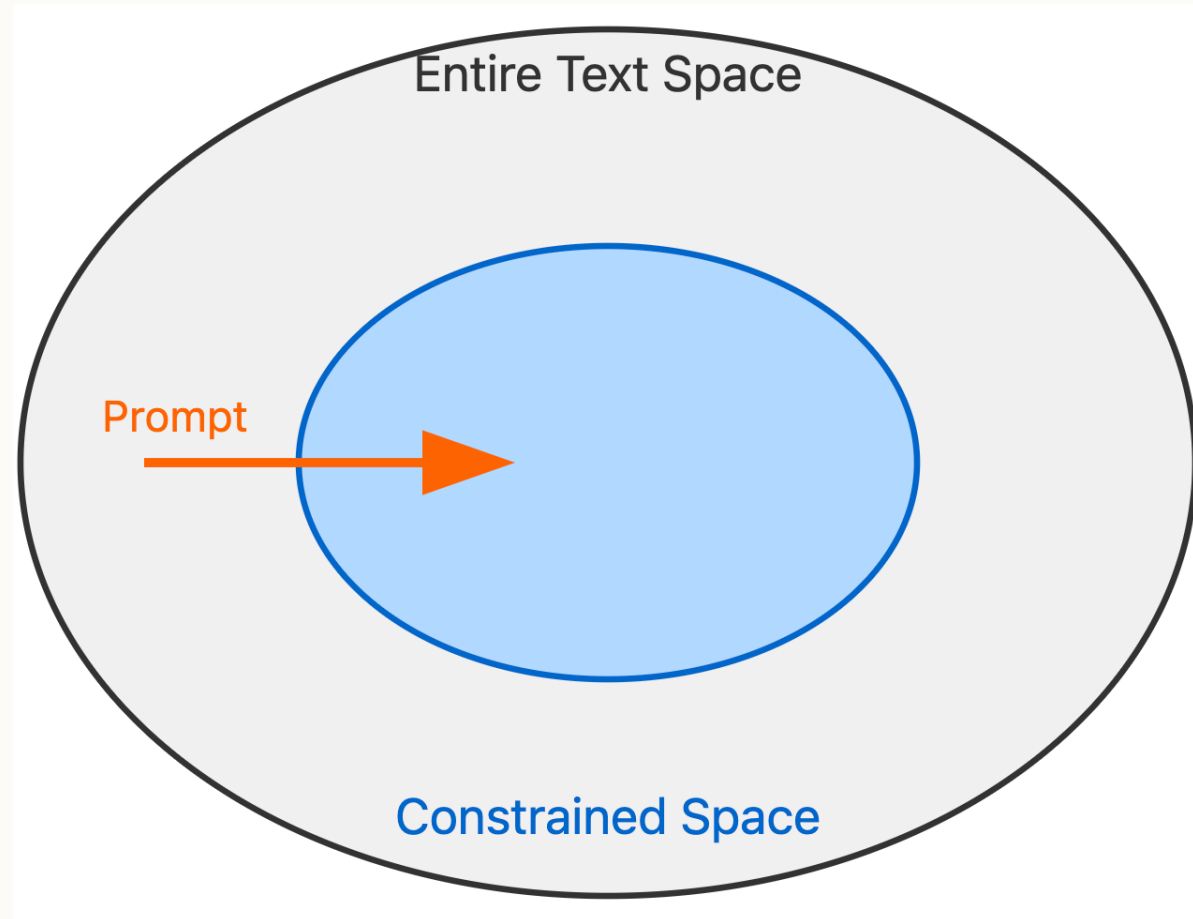
- A prompt is a piece of text that conveys to the LLM the user's intention.
- Question → Instruction → Behavior

Prompt Basics

- A prompt is a piece of text that conveys to the LLM the user's intention.
- Question → Instruction → Behavior
- It **constrains the space of possibilities** in the LLM's text space



A Prompt Constrains the Space of Potential Outputs



Prompt Basics

Components of the prompt

You are a text annotation engine.
Classify this sentence into positive or negative:
Text: *I like eating pancakes*
Output:

Prompt Basics

Components of the prompt: **instruction**

● **Instruction:** where you describe what you want

→ You are a text annotation engine.
Classify this sentence into positive or negative:
Text: *I like eating pancakes*
Output:

Prompt Basics

Components of the prompt: **instruction**, context

- **Instruction:** where you describe what you want
- **Context:** additional information to help with performance.

→ **You are a text annotation engine.**
Classify this sentence into positive or negative:
Text: *I like eating pancakes*
Output:

Prompt Basics

Components of the prompt: **instruction**, context **input data**



Instruction: where you describe what you want



Input data: example of data you wish the model to process



Context: additional information to help with performance.



You are a text annotation engine.
Classify this sentence into positive or negative:
Text: *I like eating pancakes*
Output:

Prompt Basics

Components of the prompt: **instruction**, context **input data** , **output indicator**



Instruction: where you describe what you want



Input data: example of data you wish the model to process



Context: additional information to help with performance.



Output indicator: Priming model to output what you want (e.g structure, length, etc..)

You are a text annotation engine.

Classify this sentence into positive or negative:

Text: *I like eating pancakes*

Output:



Q&A & Summary

- **Prompts** are the inputs to LLMs
- **Prompts** allow users to **guide the model's responses** and generate specific outputs.
- **Components of a prompt:** instruction, context, input data, and output indicator.
- **Responses API:** programmatic access to OpenAI models — required parameters: `model`, `input`
- **Pick the right knob:** choose the model (`gpt-5.4-mini` for most work) and the `reasoning_effort` (low → xhigh) for the task.
- **Optional Exercise During Q&A**

Create a prompt that summarizes a piece of text **in** a format/structure of your choice.

Break

Fine Tuning ChatGPT Applications

A primer on how to fine tune ChatGPT on your data



Fine Tuning:

Fine Tuning ChatGPT Applications

A primer on how to fine tune ChatGPT on your data



Fine Tuning: process of “specialising an LLM on your specific dataset”

Fine Tuning ChatGPT Applications

A primer on how to fine tune ChatGPT on your data



Fine Tuning: process of “specialising an LLM on your specific dataset”



Why?

Fine Tuning ChatGPT Applications

A primer on how to fine tune ChatGPT on your data

- **Fine Tuning:** process of “specialising an LLM on your specific dataset”
- **Why?** Better results on specific tasks, lower latency... [See OpenAI docs](#)

Fine Tuning ChatGPT Applications

A primer on how to fine tune ChatGPT on your data



Fine Tuning: process of “specialising an LLM on your specific dataset”



Why? Better results on specific tasks, lower latency... [See OpenAI docs](#)



How?

Fine Tuning ChatGPT Applications

A primer on how to fine tune ChatGPT on your data

- **Fine Tuning:** process of “specialising an LLM on your specific dataset”
- **Why?** Better results on specific tasks, lower latency... [See OpenAI docs](#)
- **How?** By preparing a custom `.jsonl` dataset on your specific use case

Fine Tuning ChatGPT Applications

A primer on how to fine tune ChatGPT on your data

- **Fine Tuning:** process of “specialising an LLM on your specific dataset”
- **Why?** Better results on specific tasks, lower latency... [See OpenAI docs](#)
- **How?** By preparing a custom `.jsonl` dataset on your specific use case

The goal of fine tuning is to adapt the model to your specific use case by providing it with a few examples of successful interactions

[7] [OpenAI's Fine Tuning Docs](#)

Fine Tuning ChatGPT Applications

Common Use Cases

Fine Tuning ChatGPT Applications

Common Use Cases



Setting style, tone,
format

[OpenAI docs](#)

Fine Tuning ChatGPT Applications

Common Use Cases



Setting style, tone,
format



Correcting failures

[OpenAI docs](#)

Fine Tuning ChatGPT Applications

Common Use Cases

- Setting style, tone, format
- Correcting failures
- Improving reliability

[OpenAI docs](#)

Fine Tuning ChatGPT Applications

Common Use Cases



Setting style, tone,
format



Handling edge cases



Correcting failures



Improving reliability

[OpenAI docs](#)

Fine Tuning ChatGPT Applications

Common Use Cases



Setting style, tone,
format



Handling edge cases



Correcting failures



Extracting structured output



Improving reliability

[OpenAI docs](#)

Fine Tuning ChatGPT Applications

Common Use Cases



Setting style, tone,
format



Correcting failures



Improving reliability



Handling edge cases



Extracting structured output



Function Calling

[OpenAI docs](#)

Fine Tuning ChatGPT Applications

Basic Steps

[OpenAI docs](#)

Fine Tuning ChatGPT Applications

Basic Steps



Prepare and upload training data

[OpenAI docs](#)

Fine Tuning ChatGPT Applications

Prepare and upload training data

```
{
  "messages": 
  [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "What's the capital of France?"},
    {"role": "assistant", "content": "Paris."}
  ]
}

{
  "messages": [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "Who wrote 'Romeo and Juliet?'"},
    {"role": "assistant", "content": "William Shakespeare."}
  ]
}

{
  "messages":
  [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "How far is the Moon from Earth?"},
    {"role": "assistant", "content": "384,400 kilometers."}
  ]
}
```

Fine Tuning ChatGPT Applications

Prepare and upload training data

```
{
  "messages":
  [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "What's the capital of France?"},
    {"role": "assistant", "content": "Paris."}
  ]
}
{
  "messages": [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "Who wrote 'Romeo and Juliet?'"},
    {"role": "assistant", "content": "William Shakespeare."}
  ]
}
{
  "messages":
  [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "How far is the Moon from Earth?"},
    {"role": "assistant", "content": "384,400 kilometers."}
  ]
}
```



Fine Tuning ChatGPT Applications

Prepare and upload training data

```
{
  "messages":
  [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "What's the capital of France?"},
    {"role": "assistant", "content": "Paris."}
  ]
}
{
  "messages": [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "Who wrote 'Romeo and Juliet?'"},
    {"role": "assistant", "content": "William Shakespeare."}
  ]
}
{
  "messages":
  [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "How far is the Moon from Earth?"},
    {"role": "assistant", "content": "384,400 kilometers."}
  ]
}
```

Fine Tuning ChatGPT Applications

Prepare and upload training data

```
{
  "messages": [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "What's the capital of France?"},
    {"role": "assistant", "content": "Paris."}
  ]
}
{
  "messages": [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "Who wrote 'Romeo and Juliet?'"},
    {"role": "assistant", "content": "William Shakespeare."}
  ]
}
{
  "messages": [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "How far is the Moon from Earth?"},
    {"role": "assistant", "content": "384,400 kilometers."}
  ]
}
```

Fine Tuning ChatGPT Applications

Prepare and upload training data

```
{
  "messages":
  [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "What's the capital of France?"},
    {"role": "assistant", "content": "Paris."}
  ]
}
{
  "messages": [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "Who wrote 'Romeo and Juliet?'"},
    {"role": "assistant", "content": "William Shakespeare."}
  ]
}
{
  "messages":
  [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "How far is the Moon from Earth?"},
    {"role": "assistant", "content": "384,400 kilometers."}
  ]
}
```



Fine Tuning ChatGPT Applications

Prepare and upload training data

```
{
  "messages":
  [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "What's the capital of France?"},
    {"role": "assistant", "content": "Paris."}
  ]
}
{
  "messages": [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "Who wrote 'Romeo and Juliet?'"},
    {"role": "assistant", "content": "William Shakespeare."}
  ]
}
{
  "messages":
  [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "How far is the Moon from Earth?"},
    {"role": "assistant", "content": "384,400 kilometers."}
  ]
}
```



Fine Tuning ChatGPT Applications

Basic Steps

- Prepare and upload training data
- Train a new fine-tuned model

[OpenAI docs](#)

Fine Tuning ChatGPT Applications

Fine Tuning Jobs

```
from openai import OpenAI
# OpenAI API key should be set as
# environment variable - OPENAI_API_KEY
client = OpenAI()
✓ client.files.create(
    file=open("dataset.jsonl", "rb"),
    purpose="fine-tune"
)
✓ client.fine_tuning.jobs.create(
    training_file="file-id",
    model="gpt-3.5-turbo-1106"
)
```

Fine Tuning ChatGPT Applications

Basic Steps

- Prepare and upload training data
- Train a new fine-tuned model
- Use your fine-tuned model

[OpenAI docs](#)

Fine Tuning ChatGPT Applications

Fine Tuning Jobs

```
def get_response(prompt, fine_tuned_model_id="ft:gpt-3.5-turbo-1106:personal::80q91T4e"):   
    client = OpenAI()  
    response = client.chat.completions.create(model=fine_tuned_model_id,  
                                              messages=  
                                              [  
                                                  {"role": "system", "content": "You are a helpful assistant."},  
                                                  {"role": "user", "content": prompt}  
                                              ],  
                                              temperature=0.0,  
                                              n = 1  
                                              )  
    return response.choices[0].message.content
```

Disclaimer: Example code is provided in the repository notebooks

Notebook Demo 3.0 - Fine Tuning OpenAI Models

LangChain **v1** + LangGraph

What changed, in three lines

LangChain v1: What Changed

- `create_agent` replaces `create_react_agent` — one standard agent built on LangGraph.
- `system_prompt` replaces the old `prompt` parameter — pass a plain string.
- Legacy chains (`LLMChain`, `RetrievalQA`, LCEL pipelines) moved to `langchain-classic` — the modern basics are **direct model calls + agents**.

```
# the whole v1 mental model in 6 lines
from langchain.agents import create_agent

agent = create_agent(
    model="openai:gpt-5.4-mini",
    tools=[my_tool],
    system_prompt="You are a helpful assistant.",
)
```

[8] [LangChain v1 Migration Guide](#)

What is LangChain?



LangChain is a framework that facilitates
creation of LLM-based applications

Main Features

Components
Off-the-shelf chains

LangChain gives you
building blocks for creating
powerful LLM applications

LangChain Building Blocks

Models

- One unified entry point for any provider: `init_chat_model`

```
from langchain.chat_models import init_chat_model

model = init_chat_model("openai:gpt-5.4-mini")

output = model.invoke("I am teaching a live-training\
    about LLMs!")

print(output.content)
```

- Swap `"openai:gpt-5.4-mini"` for `"anthropic:..."`, `"google_genai:..."` — same code.

^[9][LangChain Docs - Models](#)

LangChain Building Blocks

Messages

- Conversations are just lists of role + content — no special template language needed:

```
response = model.invoke([
    {"role": "system",
     "content": "You are a concise teaching assistant."},
    {"role": "user",
     "content": "Explain embeddings in one sentence."},
])

print(response.content)
```

- Need dynamic prompts? Plain Python f-strings work great:

```
question = f"Show me 5 examples of this concept: {concept}"
```

^[9][LangChain Docs - Messages](#)

LangChain Building Blocks

Structured Output

- Get validated Python objects instead of raw strings:

```
from pydantic import BaseModel, Field

class Quiz(BaseModel):
    question: str
    options: list[str]
    answer: str = Field(description="The correct option")

structured_model = model.with_structured_output(Quiz)

quiz = structured_model.invoke(
    "Write a quiz question about transformers")

quiz.answer  # plain attribute access, fully typed
```

^[9][LangChain Docs - Structured Output](#)

Agents with `create_agent`

Agent = Model + Tools + System Prompt

```
from langchain.agents import create_agent

def get_weather(city: str) -> str:
    """Get the current weather for a city."""
    return f"It's always sunny in {city}!"

agent = create_agent(
    model="openai:gpt-5.4-mini",
    tools=[get_weather],
    system_prompt="You are a helpful assistant.",
)

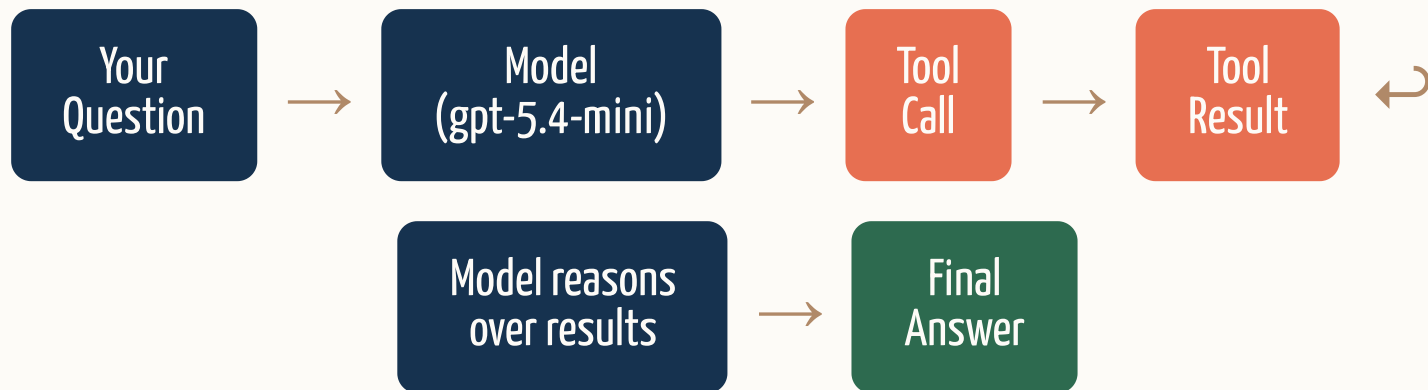
agent.invoke({"messages": [
    {"role": "user", "content": "Weather in SF?"}]})
```

- Runs on **LangGraph** under the hood → persistence, streaming, human-in-the-loop for free.

^[10] [LangChain Docs - Agents](#)

The Agent Loop

The model decides when to call your tools, observes results, and keeps going until it can answer:



- This loop is exactly what `create_agent` (LangChain) and hosted tools (OpenAI) automate for you.

Notebook Demo 4.0 - Intro to LangChain

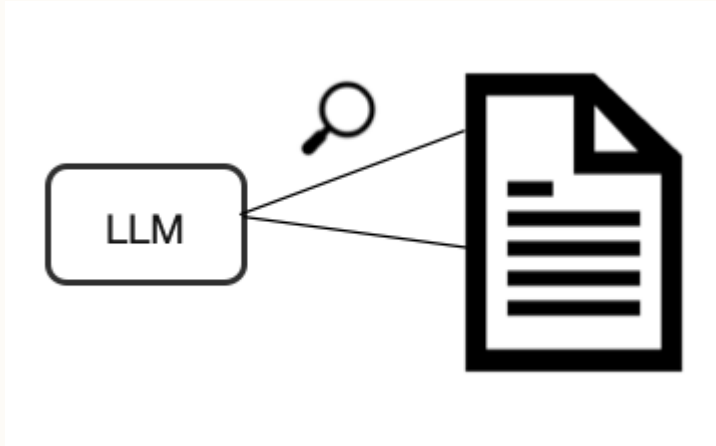
Q&A & Summary

- Fine tuning: process of training a pre-trained model on a specific task or dataset (prepare the data, train the model, use the model).
- `init_chat_model("openai:gpt-5.4-mini")` gives you one model interface across providers.
- **Messages** are plain dicts: `{"role": ..., "content": ...}` — invoke the model directly, no chain pipelines needed.
- **Structured output** with Pydantic (`with_structured_output`) turns LLM text into validated Python objects.
- `create_agent` = model + tools + `system_prompt`, running the agent loop on LangGraph.
- Optional Exercise During Q&A

Create a small agent **with** one custom tool that can answer questions about a specific topic.

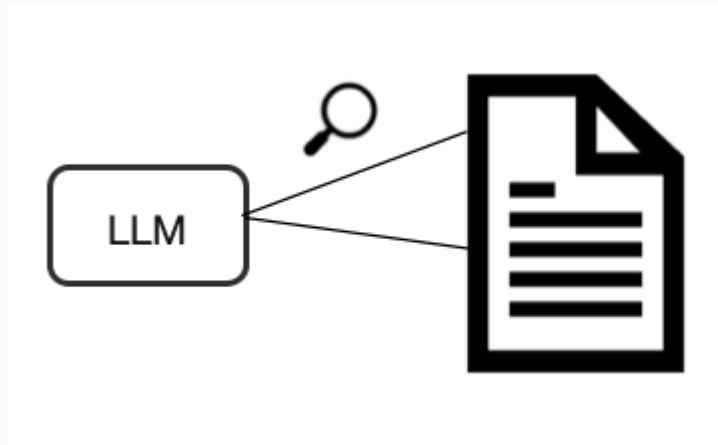
Break 5 minutes

LangChain for Chat Over Documents



- RAG = **R**etrieval **A**ugmented **G**eneration
- It's about connecting LLMs to documents like PDFs, Text files, HTML, etc.
- How do we get around the context length limitations of LLMs?
- Quick Answer is **Embeddings**!

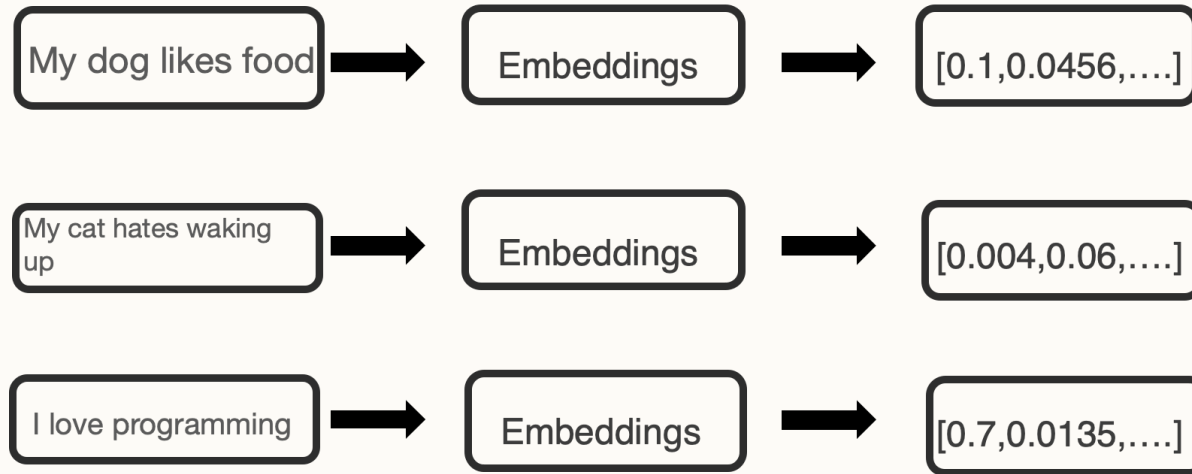
LangChain for Chat Over Documents



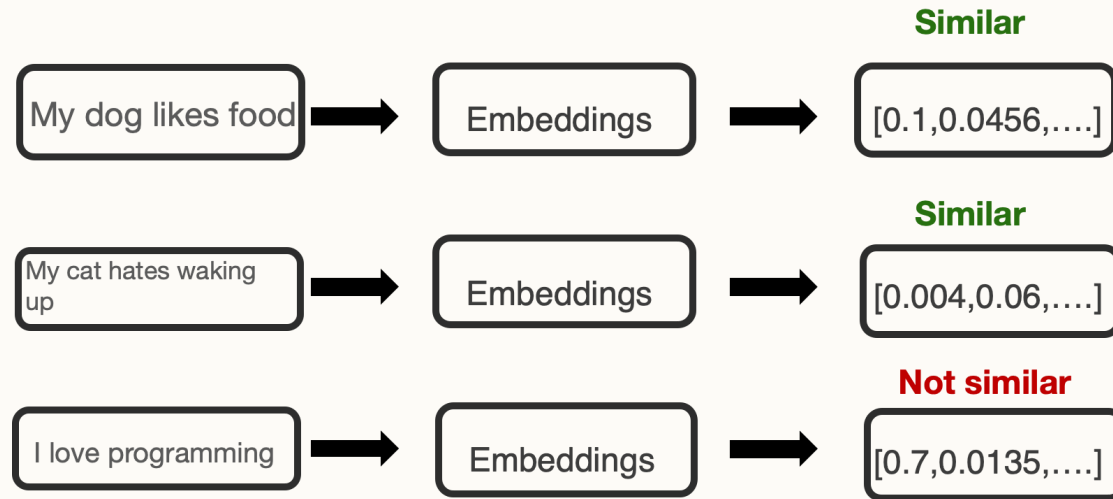
- Embeddings are vectorized representations of text



LangChain for Chat Over Documents

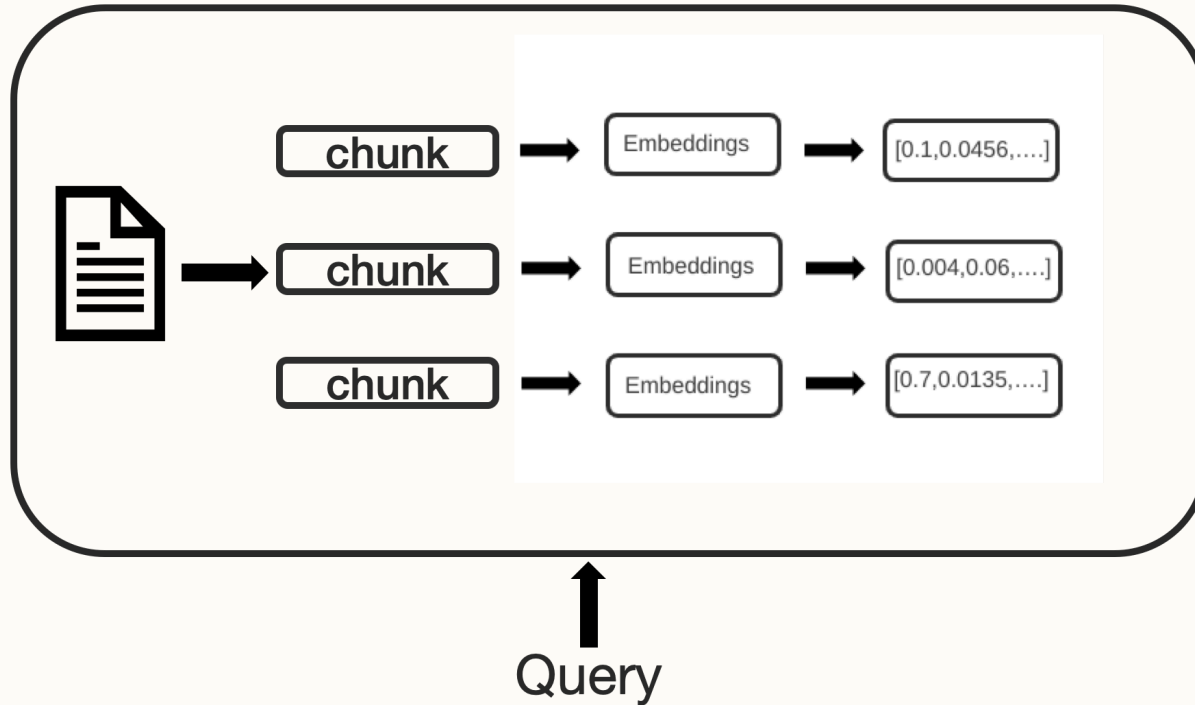


LangChain for Chat Over Documents



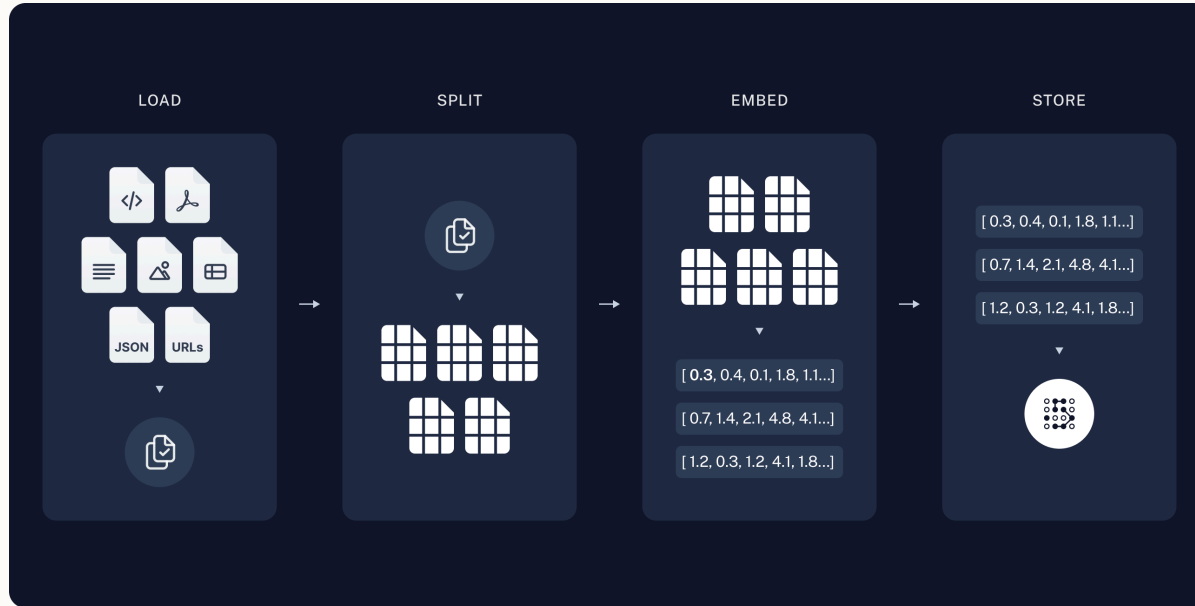
- Embeddings capture content and meaning

LangChain for Chat Over Documents



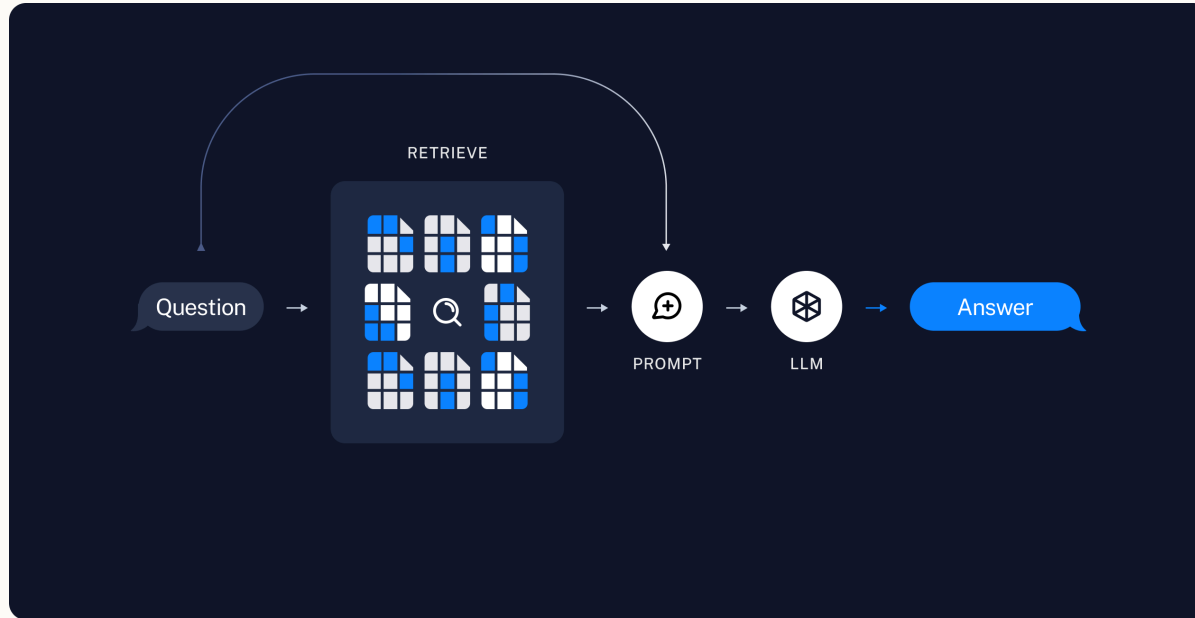
- Embeddings capture content and meaning
- Vector DBs
- How to build RAG systems with LangChain?

LangChain for Chat Over Documents



- Load
- Split
- Embed
- Store

LangChain for Chat Over Documents



- Retrieval Pipeline
- Input Question
- Retrieve Relevant Documents
- LLM uses the prompt question + retrieved data to produce a final answer

RAG the v1 Way: Retrieval as a Tool

- Sample Code

```
from langchain_openai import OpenAIEmbeddings
from langchain_chroma import Chroma
from langchain_community.document_loaders import PyPDFLoader
from langchain.agents import create_agent

docs = PyPDFLoader("path-to-pdf.pdf").load_and_split()      # LOAD + SPLIT
vectordb = Chroma.from_documents(
    docs, embedding=OpenAIEmbeddings()                      # EMBED + STORE
)

def search_pdf(query: str) -> str:
    """Search the PDF for passages relevant to the query."""
    results = vectordb.similarity_search(query, k=3)          # RETRIEVE
    return "\n\n".join(doc.page_content for doc in results)

agent = create_agent(
    model="openai:gpt-5.4-mini",
    tools=[search_pdf],
    system_prompt="Answer using the search_pdf tool.")

agent.invoke({"messages": [{"role": "user",
    "content": "What is this paper about?"}]})                # ANSWER
```

Notebook Demo 4.1 - Q&A with LangChain

Q&A & Summary

- **RAG** = Retrieval Augmented Generation
- **RAG** is about connecting LLMs to documents like PDFs, Text files, HTML, etc.
- **Embeddings** are vectorized representations of text that capture content and meaning.
- **Vector DBs** are used to store and retrieve embeddings.
- **RAG in v1**: load → split → embed → store, then expose retrieval as a **tool** to `create_agent` (no more `RetrievalQA`).
- **Optional Exercise During Q&A**

Create a simple RAG agent **with** LangChain that can answer questions about pdfs **or** csvs.

Before You Ship: What's Deprecated in 2026

Deprecated	Status	Use instead
Assistants API	Sunsets Aug 26, 2026	Responses API + Conversations
Chat Completions	Legacy (still supported)	Responses API
<code>initialize_agent</code> / <code>AgentExecutor</code>	Removed in LangChain v1	<code>create_agent</code>
<code>create_react_agent</code> (<code>langgraph.prebuilt</code>)	Replaced	<code>create_agent</code>
<code>LLMChain</code> , <code>RetrievalQA</code> , LCEL chains	Moved to <code>langchain-classic</code>	Direct <code>.invoke()</code> + agents

Rule of thumb: if a tutorial says `chat.completions`, `Assistants`, or `AgentExecutor` — it's from the past.



Migration Cheat Sheet: Old → New

You used to write...	Now write...
<code>client.chat.completions.create(...)</code>	<code>client.responses.create(...)</code>
<code>messages=[{"role": "system", ...}]</code>	<code>instructions="..." + input="..."</code>
<code>response.choices[0].message.content</code>	<code>response.output_text</code>
<code>gpt-3.5-turbo</code> / <code>gpt-4</code>	<code>gpt-5.4-mini</code> (default), <code>gpt-5.5</code> (hard stuff)
<code>ChatOpenAI(model=...)</code>	<code>init_chat_model("openai:gpt-5.4-mini")</code>
LCEL pipe chains (<code>prompt</code> , <code>llm</code> , <code>parser</code>)	<code>model.invoke(...)</code> / <code>with_structured_output</code>
<code>create_react_agent(llm, tools, prompt=...)</code>	<code>create_agent(model, tools, system_prompt=...)</code>
<code>RetrievalQA.from_llm(...)</code>	retrieval function as a tool + <code>create_agent</code>

Take a photo of this one. 📸

Practical Session

References

1. [What are Transformers and How do They Work? By Serrano Academy](#)
2. [OpenAI Docs - Responses API](#)
3. [Migrate to the Responses API](#)
4. [OpenAI Docs - Models](#)
5. [OpenAI Docs - Using GPT-5.5 & Reasoning](#)
6. [OpenAI Docs - Using Tools](#)
7. [OpenAI's Fine Tuning Docs](#)
8. [LangChain v1 Migration Guide](#)
9. [LangChain Docs - Quickstart](#)
10. [LangChain Docs - Agents](#)
11. [OpenAI Deprecations](#)
12. [ReACT Paper](#)

